Building Travelo AI

A COMPLETE GUIDE TO TECHNICAL SPECIFICATIONS, ARCHITECTURE, AND DEPLOYMENT

Travelo AI — Comprehensive Project Documentation

This document serves as the formal technical specification, architectural blueprint, and deployment guide for the Travelo AI platform.

---

1. Project Overview & Core Value Propositions

Project Overview

Travelo AI operates in the **Intelligent Travel Planning & Itinerary Generation** domain. It is designed to replace traditional, fragmented travel research (blogs, booking sites, map routing) with a unified, conversational agent. Unlike traditional chatbots, Travelo AI acts as an autonomous agentic planner that can engage in dialogue, detect missing constraints, and generate geographically optimized routes backed by real-world data.

Core Value Propositions

- **Hyper-Personalization:** Tailors every aspect of the trip based on user-specific constraints including budget, traveler type (solo, family, couple), pacing preferences (relaxed vs. packed), culinary tastes, and dietary restrictions.
- **Massive Time Savings:** Replaces hours of manual research across fragmented blogs and booking sites by instantly generating comprehensive, day-by-day itineraries.
- **Real-World Accuracy (No Hallucinations):** Unlike standard LLMs that might invent non-existent restaurants or attractions, Travelo AI uses RAG to ground all recommendations in verified, real-world data (e.g., Wikivoyage).
- **Intelligent Geographic Routing:** Eliminates the frustration of zigzagging across a city. The system clusters nearby activities and calculates optimal routes to minimize transit time.
- **Dynamic & Adaptive Planning:** Seamlessly handles complex multi-city trips and intelligently prompts the user for missing information (like travel dates or budget) instead of failing or making poor assumptions.

- **Crowd-Aware Experience:** Enhances the quality of the trip by factoring in peak crowd times, suggesting popular attractions during quieter hours.

---

2. Target Domain, Scope & Boundaries

Scope

The platform handles end-to-end trip planning:

- **Conversational Requirement Gathering:** Extracting complex user constraints (budget, dates, traveler type, pacing, cuisine).
- **Information Retrieval:** Sourcing verified, real-world data about destinations, avoiding standard LLM hallucinations.
- **Dynamic Point of Interest (POI) Scoring:** Ranking hotels, restaurants, and attractions based on custom heuristics.
- **Spatial Intelligence:** Computing physically logical routes and interleaving activities based on Haversine geographic proximity.
- **Real-Time Data Injection:** Incorporating live weather and historical crowd data (busyness scores) into the itinerary.

Boundaries (Out of Scope)

- **Direct Booking/Transactions:** The system generates itineraries and provides links but does *not* process payments or book flights/hotels directly.

---

3. Technical Stack & Core Dependencies

Travelo AI utilizes a modern, decoupled architecture designed for scale, concurrency, and rich user interactions.

Frontend (Client-Side)

- **Core Framework:** React 19 built with Vite.
- **Styling:** Tailwind CSS with custom CSS variables for dual theming (Light mode / Aurora Glassmorphism dark mode).
- **Mapping:** Leaflet & React-Leaflet for interactive geographic plotting.
- **State & Streaming:** Server-Sent Events (SSE) for token-by-token streaming of the AI's response to reduce perceived latency.
- **Authentication:** Supabase Auth (JWT).

Backend (Server-Side)

- **API Framework:** FastAPI (Python 3) for asynchronous, high-performance endpoints.
- **AI Models:**

    - **Generation Engine:** gemini-3.1-flash-lite-preview (Used for synthesizing final itineraries and answering questions).
    - **Evaluation & Intent Engine:** gemini-2.0-flash (Used for JSON-structured intent classification, coreference resolution, and the automated RAG evaluation suite).
- **Sentence Transformer:** all-MiniLM-L6-v2 (Used locally via sentence-transformers to generate text embeddings for vector storage).
- **Agent Orchestration:** LangGraph for stateful workflow management.
- **Databases:**
    - **Relational Database:** PostgreSQL (Hosted on **Supabase**, accessed via SQLAlchemy ORM). Used for storing user accounts, chat histories, and caching POIs.
    - **Vector Database:** ChromaDB. Used for storing the embedded chunks of Wikivoyage travel guides for fast semantic retrieval.
    - **In-Memory Database:** Redis. Used for high-speed rate limiting and short-TTL caching (e.g., caching Google Maps crowd data for 2 hours).
- **External APIs:**
    - **Google Gemini API:** For LLM generation.
    - **SerpAPI:** Leverages google_hotels, google_maps (for restaurants/attractions/crowds), and google_events engines.
    - **OpenWeather API:** Fetches real-time climate data via GPS coordinates.
    - **Nominatim (OpenStreetMap):** Free, open-source geocoding to translate place names into Latitude/Longitude coordinates for spatial routing.

---

4. Deep-Dive Architecture & Multi-Agent Design

The system moves beyond monolithic LLM prompts by employing a **StateGraph-based Multi-Agent Architecture** via LangGraph.

Instead of relying on a single LLM call to plan a trip, the workflow is modeled as a state machine. The TravelState (a TypedDict) holds the entire context of the conversation, the extracted parameters (budget, dates, destinations), and the aggregated POI data.

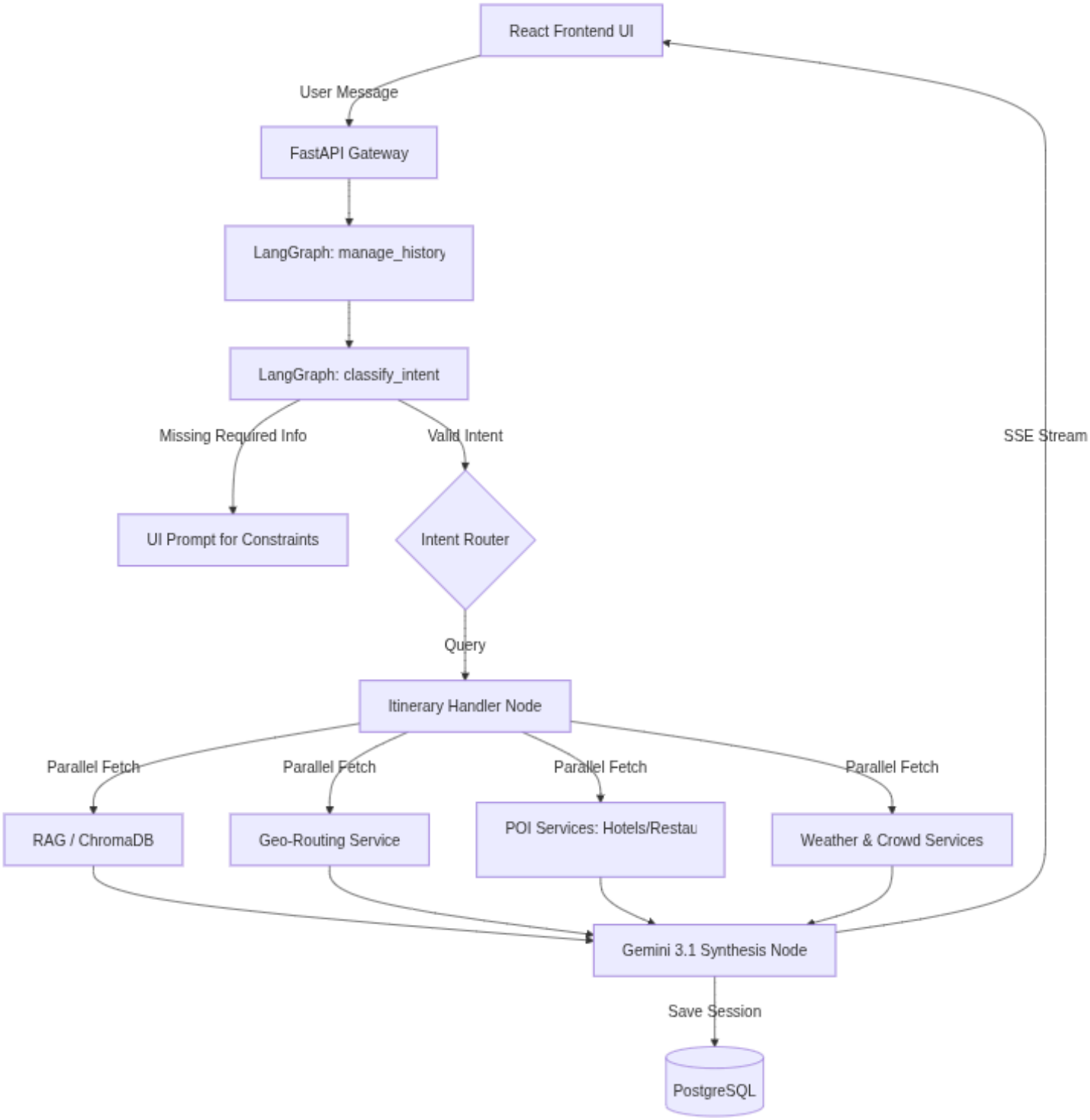
Concurrent Data Gathering:
When a valid intent is classified, the orchestrator does not block. It utilizes ThreadPoolExecutor to fire off parallel requests to the internal microservices (RAG, Hotels, Restaurants, Attractions, Weather, Crowd). This reduces the total generation time from roughly 30 seconds to under 8 seconds.

RAG (Retrieval-Augmented Generation) Pipeline:
To ensure zero hallucinations, the RAG pipeline is active. It takes the exact user query, embeds it, and queries ChromaDB. It retrieves the top 5 closest semantic chunks (derived from Wikivoyage data) and prepends them as hard factual constraints into the Gemini context window before generation occurs.

---

5. End-to-End System Flow

The following diagram illustrates the lifecycle of a user request as it travels through the multi-agent orchestration layer and interfaces with external APIs before streaming the result back to the client.



5.1 Node-by-Node Technical Specification

Setup & Routing Nodes

manage_history

This is the entry point for the system. It is responsible for managing the MemorySaver checkpointer, which ensures that the current user message is appended to the conversation history. Additionally, it clears out stale response keys from the state object to maintain an accurate and relevant dialogue flow.

classify_intent

Serving as the core decision engine, this node utilizes a structured JSON-schema Gemini 2.0 Flash call to analyze user input. It cross-references the input with the conversation history and resolves coreferences, such as mapping ambiguous phrases to previously discussed topics. If essential constraints, like budget or dates, are missing, it halts the process and returns a needs_info flag.

route (Conditional Edge)

A logic-based router that directs the state to the appropriate handler node based on the JSON intent string generated by the classify_intent node. This ensures that each user request is directed to the correct specialized handler.

Domain Handler Nodes

Depending on the identified intent, the state is directed to one of the following specialized handlers:

handle_itinerary

This complex node retrieves candidate attractions and restaurants through SerpAPI. It geocodes these candidates, organizes them using a nearest-neighbor routing algorithm from the user's origin, and interleaves meal slots based on geographic proximity. The ordered sequence is then sent to Gemini 3.1 for narrative synthesis.

handle_hotel_search

This node checks the PostgreSQL cache for recent hotel data specific to the destination. If the data is stale or missing, it performs a live SerpAPI google_hotels call based on budget and traveler type constraints.

handle_specific_hotel

Utilizes a fuzzy string match algorithm (pg_trgm similarity threshold of 0.4) against a local database to locate a specific hotel mentioned by the user. If found, it fetches its review summary; otherwise, it defaults to the handle_hotel_search node.

handle_attractions

Calls an attraction_service to fetch top-rated points of interest for the destination, attempting a local database cache hit before reaching out to external APIs.

handle_restaurants

This node engages a restaurant_service to find highly-rated dining options, filtering results by the cuisine preferences extracted from the user's intent.

handle_events

Utilizes SerpAPI google_events to identify time-sensitive local happenings, such as concerts, festivals, and sports events, in the destination area.

handle_directions

Calls the map_service to compare travel routes—driving, walking, transit—between specified start and end locations using Google Maps routing data.

handle_place_info

Delegates tasks to the RAG pipeline to embed the user's query and retrieve semantic chunks from ChromaDB, providing factual information about a destination or point of interest without generating a full itinerary.

handle_travel_question

Designed for direct logistical questions about feasibility, tips, safety, and visas, this node pulls RAG context to offer direct, bullet-pointed answers instead of generalized tourist overviews.

handle_destination_discovery

Conducts a semantic search over a vector database to recommend new travel destinations based on abstract user descriptions, such as "quiet beaches with good seafood."

handle_general_chat

A lightweight LLM wrapper that manages greetings, small talk, and expressions of gratitude with a warm, conversational travel assistant persona.

Interactive/Conversational Modifier Nodes

handle_search_add_to_itinerary

Triggered when a user wishes to dynamically insert a new place into an existing itinerary. It searches for the place, identifies it, and seeks user confirmation.

handle_confirm_add_to_itinerary

The confirmation handler that, upon user agreement, saves the pending place to the PostgreSQL SavedItem table linked to the user's account and specific itinerary day.

Finalization Node

save_response

The terminal node before the end of the interaction. It takes the response_text generated by the executed handler and appends it to the conversation_history state array, ensuring the checkpointer saves the assistant's context for future interactions.

This structured approach allows for a comprehensive and responsive travel assistant system that adapts to individual user needs and provides a seamless travel planning experience.

---

6. Sample Question & Answer Workflow

To illustrate the multi-agent architecture in action, here is the lifecycle of a single user request.

User Prompt: *"Plan a 2-day relaxed trip to Munnar for a family of 4, keeping the budget under ₹15,000. Include local vegetarian food."*

Step 1: Input & History Management

- The FastAPI endpoint receives the text. The LangGraph MemorySaver loads any previous conversation context. Since this is a new request, context is empty.

Step 2: Intent Classification

- **Gemini 2.0 Flash** analyzes the text and outputs structured JSON containing the intent (itinerary), destination (Munnar), num_days (2), budget (15000), traveler_type (family), and preferences (vegetarian food).
- Since all mandatory fields are present, it passes validation. No missing info UI popup is triggered.

Step 3: Concurrent Data Retrieval

The orchestrator fires off multiple threads simultaneously:

1. **ChromaDB (RAG):** Embeds "Munnar family trip relaxed". Retrieves 5 chunks of facts about Munnar's history and child-friendly activities from the vector store.
2. **Weather Service:** Calls OpenWeather API for Munnar's current temperature (e.g., "16°C, light rain").
3. **Attraction Service:** Calls SerpAPI for "top rated relaxed indoor family friendly kids attractions in Munnar".
4. **Hotel Service:** Calls SerpAPI for hotels under ₹15,000, boosting scores for descriptions mentioning "kids", "family", and "spacious".
5. **Restaurant Service:** Calls SerpAPI for "top rated vegetarian restaurants in Munnar".

Step 4: Geographic Routing

- The backend calculates Haversine distances between all retrieved attractions.
- It orders the day's itinerary to prevent zig-zagging and interleaves the retrieved vegetarian restaurants into the route exactly where the family will physically be during lunch times.

Step 5: Synthesis & Streaming

- All aggregated JSON data is compiled into a master prompt.
- **Gemini 3.1 Flash Lite Preview** generates the final, human-readable response, streamed token-by-token back to the React frontend via Server-Sent Events (SSE). The frontend renders the Markdown text and updates the map interface with specific POI pins.

---

7. System Evaluation & Performance Baselines

The platform includes a dedicated evaluation suite (backend/evaluation/rag_evaluator.py) to prevent regressions during prompt engineering or pipeline changes.

Current Performance Baselines (Sample Size: 20, Threshold: 0.5)

- **Faithfulness (0.92):** Highly accurate. The generated responses are strictly grounded in the retrieved facts, with minimal to zero hallucination.
- **Answer Relevancy (0.83):** Strong performance in directly answering the user's specific query without rambling.
- **Context Precision (0.81):** The chunks retrieved from ChromaDB are highly relevant to the query.
- **Context Recall (0.71):** The lowest metric, indicating occasional gaps where the database might lack the necessary depth to fully answer highly obscure niche questions (e.g., specific rare bird species in Kumarakom).

---

8. Production Cloud Deployment & Infrastructure

Travelo AI is deployed on **Google Cloud Platform (GCP)** using a highly scalable, serverless architecture.

Infrastructure Components

- **Compute:** Google Cloud Run (Fully Managed, Serverless).
  - *Backend Service:* travelo-backend (2 CPU, 2Gi Memory, Min Instances: 0).
  - *Frontend Service:* travelo-frontend (1 CPU, 256Mi Memory, Max Instances: 3).
- **Storage:**
  - Google Cloud SQL (PostgreSQL) for relational data.
  - Google Cloud Storage (GCS) bucket (travelo-chroma) mounted directly into the Cloud Run backend container to persist the ChromaDB vector files across container restarts.
- **Container Registry:** GCP Artifact Registry (asia-south1-docker.pkg.dev).

- **Secret Management:** GCP Secret Manager stores critical keys (DATABASE_URL, GEMINI_API_KEY, SERPAPI_KEY, etc.), injected securely into Cloud Run at runtime.

CI/CD Pipeline

Deployment is fully automated via **Google Cloud Build** (cloudbuild.yaml):

1. Triggered by pushes to the main branch.
2. Builds the backend Docker image and pushes it to Artifact Registry.
3. Deploys the backend to Cloud Run, retrieving the generated URL.
4. Builds the frontend Docker image, injecting the live Backend URL and Supabase credentials as build arguments (VITE_API_URL).
5. Pushes and deploys the frontend.
6. A post-deploy step automatically updates the backend CORS configuration with the newly generated frontend URL.

---

9. Conclusion

Travelo AI represents a significant leap forward in automated travel planning by successfully marrying conversational generative AI with hard, deterministic spatial routing and factual data retrieval.

By implementing an active RAG pipeline, the system dramatically reduces the severe hallucination problem that plagues general-purpose LLMs when asked for specific logistical advice. Furthermore, by utilizing a LangGraph-based multi-agent state machine, the application achieves true agentic behavior—it knows when it doesn't have enough information, actively asks the user for clarification, and concurrently orchestrates multiple APIs (weather, traffic, geographic routing, and POI sourcing) behind the scenes.

The resulting platform not only saves users hours of fragmented research but delivers highly personalized, physically logical, and reliable travel itineraries in real-time. Moving forward, the scalable GCP Cloud Run architecture and the dedicated evaluation suite ensure that Travelo AI can continue to grow, integrate live booking APIs, and refine its logic without regression, cementing its position as a next-generation AI travel companion.

Travelo AI – LangGraph Orchestrator Workflow

State Machine & Node-by-Node Flow

START

SETUP & ROUTING NODES

1. manage_history

Entry point.
Manages MemorySaver checkpointer, appends user message to conversation history and clears stale response keys from state.

2. classify_intent

Uses Gemini 2.0 Flash (JSON schema) to analyze user input + history. Resolves coreferences.
If mandatory constraints (budget, dates, etc.) are missing, sets needs_info = true and HALTS.

needs_info = true

Ask User for Missing Information

(e.g., budget, dates, travelers, etc.)

needs_info = false

3. route (Conditional Edge)

Pure logic router. Based on intent string, routes state to the appropriate handler node.

DOMAIN HANDLER NODES (Executed Based on Intent)

4. handle_itinerary

Fetches attractions & restaurants (SerpAPI). Geocodes all candidates, orders via nearest-neighbor routing from origin, interleaves meal slots by proximity. Sends sequence to Gemini 3.1 for narrative synthesis.

5. handle_hotel_search

Checks PostgreSQL cache for recent hotel data for destination. If stale or missing, makes live SerpAPI google_hotels call based on budget & traveler type constraints.

6. handle_specific_hotel

Fuzzy match (pg_trgm similarity threshold 0.4) against local DB for specific hotel. If found, fetch review summary; else automatically falls back to handle_hotel_search.

7. handle_attractions

Fetches top-rated points of interest for destination. Tries local DB cache first; falls back to external APIs (SerpAPI, OSM, etc.).

8. handle_restaurants

Fetches highly-rated dining options filtered by extracted cuisine from user intent. Uses local cache first, then SerpAPI.

9. handle_events

Checks SerpAPI google_events for time-sensitive happenings (concerts, festivals, sports) in destination area.

10. handle_directions

Uses map_service to compare routes (driving, walking, transit) between start & end locations via Google Maps Routing API.

11. handle_place_info

Delegates to RAG pipeline. Embeds query and fetches semantic chunks from ChromaDB to provide factual info about destination or POI.

12. handle_travel_question

For logistical Qs (feasibility, tips, safety, visas). Pulls RAG context but instructs LLM to provide direct, bullet-pointed answers.

13. handle_destination_discovery

Uses semantic search over vector DB to recommend new travel destinations based on abstract user descriptions (e.g., "quiet beaches with good seafood").

14. handle_general_chat

Lightweight LLM wrapper for greetings, small talk, and thank-yous with a warm, conversational travel assistant persona.

EXTERNAL SERVICES & DATA LAYER

APIs & Services

- **SerpAPI** (Search, Hotels, Events, etc.)
- **Google Maps** (Geocoding, Directions)
- **Gemini API** (LLM)
- **Other Sources** (OSM, Weather, etc.)

Local Data Layer

- **PostgreSQL DB** (Hotels, Attractions, Restaurants, Events, SavedItems, Users)
- **ChromaDB** (Vector Store for RAG)
- **MemorySaver** (Checkpointer)

INTERACTIVE / CONVERSATIONAL MODIFIER NODES

15. handle_search_add_to_itinerary

Triggered when user wants to add a new place to a generated itinerary. Searches for the place, identifies it, and asks user for confirmation.

16. handle_confirm_add_to_itinerary

Once user confirms ("yes"), saves the pending place to PostgreSQL SavedItem table linked to user account and the specific itinerary day.

LEGEND

- Setup & Routing
- Domain Handlers

FINALIZATION NODE

17. save_response

Terminal node before END.

Takes response_text from the executed handler and appends it to conversation_history state array. Ensures checkpointer saves assistant's context for the next turn.

END